CS-462 – Programming Language Foundations
Lecture #12: Simply-Typed Lambda Calculus

EPFL

A *type* is a collection of computational entities that share some common property. For example, the type **int** represents all expressions that evaluate to an integer, the type **bool** represents all expressions that evaluate to a Boolean, the type **int** $\to$ **int** represents all functions from integers to integers, and the type **int** $\times$ **bool** represents all pairs of an integer and a Boolean.

You can see types as a *static* approximation of the *dynamic* behaviors of terms and programs. Type *systems* are a lightweight formal method for reasoning about behavior of a program. Uses of type systems include: naming and organizing useful concepts; providing information (to the compiler or programmer) about data manipulated by a program; and ensuring that the run-time behavior of programs meet certain criteria.

In this lecture, we'll consider a type system for the lambda calculus that ensures that values are used correctly; for example, that a program never tries to add an integer to a function. The resulting language (lambda calculus plus the type system) is called the *simply-typed lambda calculus* (STLC).

1 Simply-typed lambda calculus

The syntax of the simply-typed lambda calculus is similar to that of untyped lambda calculus, with the exception of abstractions. Since abstractions define functions that take an argument, in the simply-typed lambda calculus, we explicitly state what the type of the argument is. That is, in an abstraction $\lambda x : \tau.\, e$, the τ is the expected type of the argument.

The syntax of the simply-typed lambda calculus is as follows. It includes integer literals n, arithmetic $e_1 \oplus e_2$ (where the operator $\oplus$ is addition, subtraction, or multiplication), and the *unit value* $()$. The unit value is the only value of type **unit**.

expressions	$e ::= x \mid \lambda x : \tau.\, e \mid e_1\ e_2 \mid n \mid e_1 \oplus e_2 \mid ()$
operators	$\oplus ::= + \mid - \mid \times$
values	$v ::= \lambda x : \tau.\, e \mid n \mid ()$
types	$\tau ::=$ **int** $\mid$ **unit** $\mid \tau_1 \to \tau_2$

The operational semantics of the simply-typed lambda calculus are the same as the untyped lambda calculus. For completeness, we present the CBV small step operational semantics here.

$$E ::= [\cdot] \mid E\ e \mid v\ E \mid E \oplus e \mid v \oplus E$$

$$\textsc{Context}\ \frac{e \to e'}{E[e] \to E[e']}$$

$$\beta\textsc{-reduction}\ \frac{}{(\lambda x : \tau.\, e)\ v \to e\{v/x\}}$$

$$\textsc{Arith}\ \frac{n = n_1 \oplus n_2}{n_1 \oplus n_2 \to n}$$

1.1 The typing relation

The presence of types does not alter the evaluation of an expression at all. So what use are types?

We will use types to restrict what expressions we will evaluate. Specifically, the type system for the simply-typed lambda calculus will ensure that any *well-typed* program will not get *stuck*. A term e is stuck if e is not a value and there is no term e' such that $e \longrightarrow e'$. For example, the expression $42 + \lambda x.\, x$ is stuck: it attempts to add an integer and a function; it is not a value, and there is no operational rule that allows us to reduce this expression. Another stuck expression is $()\ 47$, which attempts to apply the unit value to an integer.

We introduce a relation (or *judgment*) over *typing contexts* (or *type environments*) Γ, expressions e, and types τ. The judgment

$$\Gamma \vdash e : \tau$$

is read as "e has type τ in context Γ".

A typing context is a sequence of variables and their types. In the typing judgment $\Gamma \vdash e : \tau$, we will ensure that if x is a free variable of e, then Γ associates x with a type. We can view a typing context as a partial function from variables to types. We will write $\Gamma, x : \tau$ or $\Gamma[x \mapsto \tau]$ to indicate the typing context that extends Γ by associating variable x with type τ. The empty context is sometimes written $\emptyset$, or often just not written at all. For example, we write $\vdash e : \tau$ to mean that the closed term e has type τ under the empty context.

Given a typing environment Γ and expression e, if there is some τ such that $\Gamma \vdash e : \tau$, we say that *e is well-typed under context* Γ; if Γ is the empty context, we say *e is well-typed*.

We define the judgment $\Gamma \vdash e : \tau$ inductively.

$$\text{T-Int}\ \frac{}{\Gamma \vdash n : \mathbf{int}} \qquad \text{T-Arith}\ \frac{\Gamma \vdash e_1 : \mathbf{int} \quad \Gamma \vdash e_2 : \mathbf{int}}{\Gamma \vdash e_1 \oplus e_2 : \mathbf{int}} \qquad \text{T-Unit}\ \frac{}{\Gamma \vdash () : \mathbf{unit}}$$

$$\text{T-Var}\ \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \qquad \text{T-Abs}\ \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x : \tau.\, e : \tau \to \tau'} \qquad \text{T-App}\ \frac{\Gamma \vdash e_1 : \tau \to \tau' \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1\ e_2 : \tau'}$$

An integer n always has type **int**. Expression $e_1 \oplus e_2$ has type **int** if both e_1 and e_2 have type **int**. The unit value $()$ always has type **unit**.

Variable x has whatever type the context associates with x. Note that Γ must contain an association for x in order for the judgment $\Gamma \vdash x : \tau$ to hold, that is, $x \in dom(\Gamma)$. The abstraction $\lambda x : \tau.\, e$ has the function type $\tau \to \tau'$ if the function body e has type τ' under the assumption that x has type τ. Finally, an application $e_1\ e_2$ has type τ' provided that e_1 is a function of type $\tau \to \tau'$, and e_2 is an argument of the expected type, i.e., of type τ.

To type check an expression e, we attempt to construct a derivation of the judgment $\vdash e : \tau$, for some type τ. For example, consider the program $(\lambda x : \mathbf{int}.\, x + 40)\ 2$. The following is a proof that $(\lambda x : \mathbf{int}.\, x + 40)\ 2$ is well-typed.

$$\text{T-App}\ \frac{\text{T-Abs}\ \dfrac{\text{T-Arith}\ \dfrac{\text{T-Var}\ \dfrac{}{x : \mathbf{int} \vdash x : \mathbf{int}} \quad \text{T-Int}\ \dfrac{}{x : \mathbf{int} \vdash 40 : \mathbf{int}}}{x : \mathbf{int} \vdash x + 40 : \mathbf{int}}}{\vdash \lambda x : \mathbf{int}.\, x + 40 : \mathbf{int} \to \mathbf{int}} \quad \text{T-Int}\ \dfrac{}{\vdash 2 : \mathbf{int}}}{\vdash (\lambda x : \mathbf{int}.\, x + 40)\ 2 : \mathbf{int}}$$

1.2 Type soundness

We mentioned above that the type system ensures that any well-typed program does not get stuck. We can state this property formally.

Theorem (Type soundness)**.** *If* $\vdash e : \tau$ *and* $e \to^* e'$ *and* $e' \not\to$ *then* e' *is a value and* $\vdash e' : \tau$.

We will prove this theorem using two lemmas: *preservation* and *progress*. Intuitively, preservation says that if an expression e is well-typed, and e can take a step to e', then e' is well-typed. That is, evaluation preserves well-typedness. Progress says that if an expression e is well-typed, then either e is a value, or there is an e' such that e can take a step to e'. That is, well-typedness means that the expression cannot get stuck.

Together, these two lemmas suffice to prove type soundness. Given the preservation lemma, a trivial induction on the number of steps taken to reach e' from e establishes that $\vdash e' : \tau$. Then the progress lemma ensures that, if e' cannot take a step, then it must be a value.

1.3 Preservation

To prove preservation, we will need some extra tiny lemmas.

Lemma (Substitution)**.** *If* $\Gamma, x : \tau' \vdash e : \tau$ *and* $\vdash v : \tau'$ *then* $\Gamma \vdash e\{v/x\} : \tau$.

Lemma (Context)**.** *If* $\vdash E[e] : \tau$*, where the derivation of this judgment gives* e *the type* τ'*, and* $\vdash e' : \tau'$*, then* $\vdash E[e'] : \tau$.

We'll assume these without proof. (They're not difficult, but the proof of substitution can get rather long. It is by induction on the typing derivation, which is why the lemma is stated for an arbitrary context Γ: the case for $\lambda y : \tau_1. e_1$ needs the induction hypothesis for the body e_1, in the larger context $\Gamma, y : \tau_1$. Preservation only uses the case where Γ is empty.) Equipped with these little lemmas, we're ready to move on to the main proof of preservation.

A quick note on proof strategy: to prove preservation, it's possible to induct either on the typing relation or on the small-step relation. Both have their advantages and disadvantages; we'll use the small-step relation here.

Lemma (Preservation)**.** *If* $\vdash e : \tau$ *and* $e \to e'$ *then* $\vdash e' : \tau$.

Proof. Assume $\vdash e : \tau$ and $e \to e'$. We need to show $\vdash e' : \tau$. We will do this by induction on the derivation of $e \to e'$.

- ARITH

  Here, $e \equiv n_1 \oplus n_2$, and $e' = n$ where $n = n_1 \oplus n_2$.

  There is only one typing rule that applies to arithmetic expressions, T-ARITH, from which we know $\tau = \textbf{int}$.

  By the typing rule T-INT, we have $\vdash e' : \textbf{int}$ as required.

- β-REDUCTION

  Here, $e \equiv (\lambda x : \tau'. e_1)\ v$ and $e' \equiv e_1\{v/x\}$.

  Since e is well-typed by assumption, we have derivations showing $\vdash \lambda x : \tau'. e_1 : \tau' \to \tau$ and $\vdash v : \tau'$. There is only one typing rule for abstractions, T-ABS, from which we know $x : \tau' \vdash e_1 : \tau$.

  By our substitution lemma above, we have $\vdash e_1\{v/x\} : \tau$ as required.

- CONTEXT

  Here, we have some context E such that $e = E[e_1]$ and $e' = E[e_2]$ for some e_1 and e_2 such that $e_1 \longrightarrow e_2$.

  Since e is well-typed, we can show by induction on the structure of E that $\vdash e_1 : \tau_1$ for some τ_1. (This simple sub-induction is left as an exercise.)

  By the induction hypothesis and because we know $e_1 \longrightarrow e_2$, we have $\vdash e_2 : \tau_1$. (Put intuitively, e_2 has the same type as the one we just established for e_1.)

  By our context lemma above, we have $\vdash E[e_2] : \tau$ as required.

□

1.4 Progress

To prove our progress lemma, we'll need one extra lemma that gives us the syntax forms for closed terms.

Lemma (Canonical Forms)**.** *If $\vdash v : \tau$, then*

1. *If τ is **int**, then v is an integer literal, i.e., some n.*
2. *If τ is $\tau_1 \longrightarrow \tau_2$, then v is an abstraction, i.e., $\lambda x : \tau_1 . e$ for some x and e.*

Proof. The proof is by inspection of the typing rules.

i If τ is int, then the only rule which lets us give a value this type is T-INT.

ii If τ is $\tau_1 \longrightarrow \tau_2$, then the only rule which lets us give a value this type is T-ABS.

□

Now we're ready to prove progress.

Lemma (Progress)**.** *If $\vdash e : \tau$ then either e is a value or there exists an e' such that $e \longrightarrow e'$.*

Proof. We proceed by induction on the derivation of $\vdash e : \tau$.

- T-VAR

  This case is impossible, since a variable is not well-typed in the empty environment.

- T-UNIT, T-INT, T-ABS

  In all of these cases, e is a value.

- T-ARITH

  Here $e \equiv e_1 \oplus e_2$ and $\vdash e_1 : \mathbf{int}$ and $\vdash e_2 : \mathbf{int}$. By the inductive hypothesis, for $i \in \{1, 2\}$ (i.e., for both e_1 and e_2), either e_i is a value or there is an e_i' such that $e_i \longrightarrow e_i'$.

  If e_1 is not a value, we have from above that $e_1 \longrightarrow e_1'$. Therefore, by the CONTEXT rule, $e_1 \oplus e_2 \longrightarrow e_1' \oplus e_2$.

  Otherwise, e_1 is a value. If e_2 is not a value, then by CONTEXT again, $e_1 \oplus e_2 \longrightarrow e_1 \oplus e_2'$.

  Otherwise, both e_1 and e_2 are values. By our canonical forms lemma, $e_1 = n_1$ and $e_2 = n_2$ are both integer literals. By the ARITH rule, we have $e_1 \oplus e_2 \longrightarrow n$ where $n = n_1 \oplus n_2$.

- T-App

  Here $e \equiv e_1\ e_2$ and $\vdash e_1 : \tau' \to \tau$ and $\vdash e_2 : \tau'$. By the inductive hypothesis, for $i \in \{1, 2\}$, either e_i is a value or there is an e_i' such that $e_i \to e_i'$.

  If e_1 is not a value, then by the above and by applying the Context rule, $e_1\ e_2 \to e_1'\ e_2$.

  Otherwise, e_1 is a value. If e_2 is not a value, then by Context, $e_1\ e_2 \to e_1\ e_2'$.

  If e_1 and e_2 are values, then, by our canonical forms lemma, e_1 is an abstraction $\lambda x : \tau'.\, e'$. Therefore, by β-reduction, we have $e_1\ e_2 \to e'\{e_2/x\}$.

□

1.5 Uniqueness of Types

Our typing rules have another useful property: every well-typed expression has exactly one type.

Lemma (Uniqueness of types)**.** *If* $\Gamma \vdash e : \tau$ *and* $\Gamma \vdash e : \tau'$ *then* $\tau = \tau'$.

Proof sketch. By induction on e. The typing rules are *syntax-directed*: for each form of expression, exactly one typing rule can be used to conclude its type. For example, if $e \equiv \lambda x : \tau_1.\, e_1$, then both derivations must end with T-Abs, so $\tau = \tau_1 \to \tau_2$ and $\tau' = \tau_1 \to \tau_2'$, where $\Gamma, x : \tau_1 \vdash e_1 : \tau_2$ and $\Gamma, x : \tau_1 \vdash e_1 : \tau_2'$. By the induction hypothesis, $\tau_2 = \tau_2'$. Note that the annotation τ_1 on the abstraction is essential here. The other cases are similar. □

Because the rules are syntax-directed, they also give a simple algorithm for type checking: a recursive function over the syntax of e that computes the unique type of e, or reports that e is not well-typed.

1.6 Type Completeness?

Not all expressions in the untyped lambda calculus are well-typed. Type soundness implies that any lambda calculus program that gets stuck is not well-typed.

But are there programs that *do not* get stuck that are not well-typed? In other words, does our type system unjustly rule out legal programs?

Unfortunately, the answer is yes. In particular, because the simply-typed lambda calculus requires us to specify a type for function arguments, any given function can only take arguments of one type. Consider, for example, the identity function $\lambda x.\, x$. This function may be applied to any argument, and it will not get stuck. However, we must provide a type for the argument. If we specify $\lambda x : \mathbf{int}.\, x$, then this function can only accept integers, and the program $(\lambda x : \mathbf{int}.\, x)\ ()$ is not well-typed, even though it does not get stuck. Indeed, in the simply-typed lambda calculus, there is a different identity function for each type.

1.7 Termination

The type system also rules out some programs that do not get stuck but instead run forever. Consider $\Omega \triangleq (\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$. To write Ω in the simply-typed lambda calculus, we would need a type τ such that $\lambda x : \tau.\, x\ x$ is well-typed. By T-App, the application $x\ x$ requires x to have a function type $\tau_1 \to \tau_2$, and also to have the argument type τ_1. By T-Var, both of these are τ, so

$$\tau_1 = \tau = \tau_1 \to \tau_2.$$

But no type satisfies this equation, since τ_1 would have to be a proper part of itself, and types are finite. So Ω is not well-typed, and neither are the fixed-point combinators Y and Z, which use self-application in the same way.

In fact, every well-typed program terminates.

Theorem (Strong normalization)**.** *If $\vdash e : \tau$, then every sequence of reductions starting from e is finite.*

The proof uses a technique called *logical relations*, due to Tait, which we will not cover here. One consequence is that the simply-typed lambda calculus is not Turing complete: since every well-typed program terminates, it cannot express all computable functions, and in particular it cannot express a function that runs forever on some inputs. This is another way in which the type system is incomplete. To recover general recursion, we will add a fixed-point operator to the language as a primitive below.

2 Advanced Types

In this lecture we will extend the simply-typed λ-calculus with several features we saw earlier in the course, including products and sums, as well as some new ones.

2.1 Booleans and Conditionals

We add the Boolean constants true and false, conditionals, and a comparison operator on integers.

$$e ::= \cdots \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 \mid e_1 = e_2$$
$$v ::= \cdots \mid \mathsf{true} \mid \mathsf{false}$$

The evaluation contexts evaluate the guard of a conditional, but neither branch: only the branch that is selected is evaluated.

$$E ::= \cdots \mid \mathsf{if}\ E\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 \mid E = e \mid v = E$$

$$\frac{}{\mathsf{if\ true\ then}\ e_2\ \mathsf{else}\ e_3 \longrightarrow e_2} \qquad \frac{}{\mathsf{if\ false\ then}\ e_2\ \mathsf{else}\ e_3 \longrightarrow e_3}$$

$$\frac{n_1 = n_2}{n_1 = n_2 \longrightarrow \mathsf{true}} \qquad \frac{n_1 \neq n_2}{n_1 = n_2 \longrightarrow \mathsf{false}}$$

We add a new type **bool** of Booleans.

$$\tau ::= \cdots \mid \mathbf{bool}$$

The typing rules are as follows. Both branches of a conditional must have the same type, which is the type of the whole conditional.

$$\text{T-True}\ \frac{}{\Gamma \vdash \mathsf{true} : \mathbf{bool}} \qquad \text{T-False}\ \frac{}{\Gamma \vdash \mathsf{false} : \mathbf{bool}}$$

$$\text{T-If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau}$$

$$\text{T-Eq}\ \frac{\Gamma \vdash e_1 : \mathbf{int} \quad \Gamma \vdash e_2 : \mathbf{int}}{\Gamma \vdash e_1 = e_2 : \mathbf{bool}}$$

2.2 What Goes Wrong?

It is instructive to see how the type soundness proof fails if we get the typing rules wrong. Suppose we changed T-If so that the type of a conditional is the type of its first branch, and the second branch may have any type:

$$\text{T-If?}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau'}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau}$$

Then preservation fails: $\vdash \mathsf{if}\ \mathsf{false}\ \mathsf{then}\ 1\ \mathsf{else}\ () : \mathbf{int}$, but it steps to $()$, which has type **unit**. As a result, soundness fails as well: $(\mathsf{if}\ \mathsf{false}\ \mathsf{then}\ 1\ \mathsf{else}\ ()) + 1$ has type **int**, but it steps to $() + 1$, which is stuck.

Now suppose instead that we allowed the first argument of addition to be a Boolean:

$$\text{T-Plus?}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \quad \Gamma \vdash e_2 : \mathbf{int}}{\Gamma \vdash e_1 + e_2 : \mathbf{int}}$$

Then progress fails: $\vdash \mathsf{true} + 1 : \mathbf{int}$, but $\mathsf{true} + 1$ is stuck, because no evaluation rule adds a Boolean to an integer.

Mistakes in the operational semantics can break soundness too. For example, if we forgot the evaluation context $v \oplus E$, then $1 + (2 + 3)$ would be well-typed but stuck: 1 is already a value, so the context $E \oplus e$ does not apply, and there is no way to evaluate $2 + 3$. In general, the typing rules, the evaluation rules, and the proofs of progress and preservation must all fit together.

2.3 Products

We have previously seen how to encode *products* into untyped λ-calculus.

$$\begin{aligned} e &::= \cdots \mid (e_1, e_2) \mid \#1\ e \mid \#2\ e \\ v &::= \cdots \mid (v_1, v_2) \end{aligned}$$

We defined congruence rules that determine the order of evaluation, using the following evaluation contexts.

$$E ::= \cdots \mid (E, e) \mid (v, E) \mid \#1\ E \mid \#2\ E$$

We also defined two computation rules that determine how the pairing constructor and destructors interact.

$$\frac{}{\#1\ (v_1, v_2) \to v_1} \qquad \frac{}{\#2\ (v_1, v_2) \to v_2}$$

In simply-typed λ-calculus, the type of a product expression (or a *product type*) is a pair of types, written $\tau_1 \times \tau_2$. The typing rules for the product constructors and destructors are as follows:

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \#1\ e : \tau_1} \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \#2\ e : \tau_2}$$

Note the similarities between these rules and the proof rules for conjunction in natural deduction. We will examine this relationship closely later in the course.

2.4 Derived Forms

Some useful constructs can be defined in terms of the ones we already have. These *derived forms* (or *syntactic sugar*) need no new evaluation or typing rules. For example, we can define local definitions and sequencing:

$$\begin{aligned} \mathsf{let}\ x : \tau = e_1\ \mathsf{in}\ e_2 &\triangleq (\lambda x : \tau.\, e_2)\ e_1 \\ e_1;\ e_2 &\triangleq (\lambda x : \mathbf{unit}.\, e_2)\ e_1 \qquad \text{where } x \notin \mathit{fv}(e_2) \end{aligned}$$

Under CBV, $\mathsf{let}\ x : \tau = e_1\ \mathsf{in}\ e_2$ evaluates e_1 to a value and then substitutes it for x in e_2, and $e_1;\ e_2$ evaluates e_1, discards its value, and then evaluates e_2. Their typing rules can be derived from T-Abs and T-App:

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma, x : \tau \vdash e_2 : \tau'}{\Gamma \vdash \mathsf{let}\ x : \tau = e_1\ \mathsf{in}\ e_2 : \tau'} \qquad \frac{\Gamma \vdash e_1 : \mathbf{unit} \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1;\ e_2 : \tau}$$

For example, the first rule follows by applying T-Abs to the second premise, which gives $\Gamma \vdash \lambda x : \tau.\, e_2 : \tau \to \tau'$, and then T-App with the first premise. The second rule is similar, using the fact that adding $x : \mathbf{unit}$ to Γ does not change the type of e_2, since x is not free in e_2. We often omit the annotation on let, since τ must be the type of e_1.

2.5 Sums

Our next example, *sums*, is dual to products. Intuitively, a product holds two values, one of type τ_1, and one of type τ_2, while a sum holds a single value that is either of type τ_1 or of type τ_2. The type of a sum is written $\tau_1 + \tau_2$. There are two constructors for sums, corresponding to whether we are constructing a sum with a value of τ_1 or a value of τ_2.

$$\begin{aligned} e &::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\ e \mid \mathsf{inr}_{\tau_1+\tau_2}\ e \mid \mathsf{case}\ e_1\ \mathsf{of}\ e_2 \mid e_3 \\ v &::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\ v \mid \mathsf{inr}_{\tau_1+\tau_2}\ v \end{aligned}$$

There are congruence rules that determine the order of evaluation, as defined by the following evaluation contexts.

$$E ::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\ E \mid \mathsf{inr}_{\tau_1+\tau_2}\ E \mid \mathsf{case}\ E\ \mathsf{of}\ e_2 \mid e_3$$

There are also two computation rules that show how the constructors and destructors interact.

$$\frac{}{\mathsf{case}\ \mathsf{inl}_{\tau_1+\tau_2}\ v\ \mathsf{of}\ e_2 \mid e_3 \longrightarrow e_2\ v} \qquad \frac{}{\mathsf{case}\ \mathsf{inr}_{\tau_1+\tau_2}\ v\ \mathsf{of}\ e_2 \mid e_3 \longrightarrow e_3\ v}$$

The type of a sum expression (or a *sum type*) is written $\tau_1 + \tau_2$. The typing rules for the sum constructors and destructor are the following.

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \mathsf{inl}_{\tau_1+\tau_2}\ e : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \mathsf{inr}_{\tau_1+\tau_2}\ e : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad \Gamma \vdash e_1 : \tau_1 \to \tau \quad \Gamma \vdash e_2 : \tau_2 \to \tau}{\Gamma \vdash \mathsf{case}\ e\ \mathsf{of}\ e_1 \mid e_2 : \tau}$$

Let's see an example of a program that uses sum types.

$$\begin{array}{l}\mathsf{let}\ f = \lambda a : \mathbf{int} + (\mathbf{int} \to \mathbf{int}).\ \mathsf{case}\ a\ \mathsf{of}\ (\lambda y : \mathbf{int}.\ y + 1) \mid (\lambda g : \mathbf{int} \to \mathbf{int}.\ g\ 35)\ \mathsf{in} \\ \mathsf{let}\ h = \lambda x : \mathbf{int}.\ x + 7\ \mathsf{in} \\ f\ (\mathsf{inr}_{\mathbf{int}+(\mathbf{int}\to\mathbf{int})}\ h)\end{array}$$

The function f takes argument a, which is a sum—that is, the actual argument for a will either be a value of type **int** or a value of type **int** → **int**. We destruct the sum value with a case statement, which must be prepared to take either of the two kinds of values that the sum may contain. In this instance, we end up applying f to a value of type **int** → **int** (i.e., a value injected into the right type of the sum), so the entire program ends up evaluating to 42.

2.6 Extending the Proofs

When we extend the language, we must also extend the proof of type soundness. Most of the work is in the canonical forms lemma, which gains a case for each new type of values.

Lemma (Canonical Forms, continued)**.** *If* $\vdash v : \tau$, *then*

1. *If* τ *is* ***bool****, then* v *is* true *or* false.
2. *If* τ *is* $\tau_1 \times \tau_2$, *then* v *is a pair* (v_1, v_2) *with* $\vdash v_1 : \tau_1$ *and* $\vdash v_2 : \tau_2$.
3. *If* τ *is* $\tau_1 + \tau_2$, *then* v *is* $\mathsf{inl}_{\tau_1+\tau_2}\ v'$ *or* $\mathsf{inr}_{\tau_1+\tau_2}\ v'$.

Each new typing rule then adds a case to the proofs of preservation and progress. For example, here is the case of the progress proof for conditionals.

- T-If

  Here $e \equiv \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3$ and $\vdash e_1 : \mathbf{bool}$. By the inductive hypothesis, either e_1 is a value or there is an e_1' such that $e_1 \to e_1'$.

  If $e_1 \to e_1'$, then by the Context rule with the context $\mathsf{if}\ [\cdot]\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3$, we have $e \to \mathsf{if}\ e_1'\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3$.

  Otherwise, e_1 is a value, so by canonical forms it is true or false, and e steps to e_2 or e_3 by one of the evaluation rules for conditionals.

Note that we did not need the inductive hypothesis for e_2 or e_3, because neither branch is evaluated before the guard. The corresponding cases of preservation follow directly from T-If: if $e \to e_2$, then $\vdash e_2 : \tau$ is one of its premises, and similarly for e_3.

2.7 Fixed Points

As we saw above, fixed-point combinators such as Y and Z are not well-typed, and every well-typed program terminates. To obtain fixed points in the simply-typed lambda calculus, we can instead add a new primitive fix to the language. The evaluation rules for the new primitive mimic the behavior of the fixed-point combinators we saw previously.

We extend the syntax with the new primitive operator. Intuitively, $\mathsf{fix}\ e$ is the fixed-point of the function e. Note that $\mathsf{fix}\ v$ is *not* a value.

$$e ::= \cdots \mid \mathsf{fix}\ e$$

We extend the operational semantics for the new operator. There is a new evaluation context, and a new axiom.

$$E ::= \cdots \mid \mathsf{fix}\ E \qquad\qquad \frac{}{\mathsf{fix}\ \lambda x:\tau.\, e \longrightarrow e\{(\mathsf{fix}\ \lambda x:\tau.\, e)/x\}}$$

Note that we can define the $\mathsf{letrec}\ x:\tau = e_1\ \mathsf{in}\ e_2$ construct in terms of the fix operator.

$$\mathsf{letrec}\ x:\tau = e_1\ \mathsf{in}\ e_2 \triangleq \mathsf{let}\ x = \mathsf{fix}\ \lambda x:\tau.\, e_1\ \mathsf{in}\ e_2$$

The typing rule for fix says that $\mathsf{fix}\ e$ is a fixed point of a function e from τ to τ, so it has type τ:

$$\textsc{T-Fix}\ \frac{\Gamma \vdash e : \tau \longrightarrow \tau}{\Gamma \vdash \mathsf{fix}\ e : \tau}$$

Returning to our trusty factorial example, the following program implements the factorial function using the fix operator.

$$\mathsf{FACT} \triangleq \mathsf{fix}\ \lambda f : \mathbf{int} \longrightarrow \mathbf{int}.\ \lambda n : \mathbf{int}.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n \times (f\ (n-1))$$

Note that we can write non-terminating computations for any type: the expression $\mathsf{fix}\ \lambda x:\tau.\, x$ has type τ, and does not terminate.

Although the fix operator is normally used to define recursive functions, it can be used to find fixed points of any type. For example, consider the following expression.

$$\begin{aligned}\mathsf{fix}\ \lambda x : (\mathbf{int} \longrightarrow \mathbf{bool}) \times (\mathbf{int} \longrightarrow \mathbf{bool}).\ (&\lambda n : \mathbf{int}.\ \mathsf{if}\ n = 0\ \mathsf{then}\ \mathsf{true}\ \mathsf{else}\ (\#2\ x)\ (n-1),\\ &\lambda n : \mathbf{int}.\ \mathsf{if}\ n = 0\ \mathsf{then}\ \mathsf{false}\ \mathsf{else}\ (\#1\ x)\ (n-1))\end{aligned}$$

This expression defines a pair of mutually recursive functions; the first function returns true if and only if its argument is even; the second function returns true if and only if its argument is odd.

2.8 Exceptions

Many programming languages provide support for throwing and catching exceptions. We can model an extremely simple form of exceptions by extending the simply-typed λ-calculus with a single exception representing an error. We first extend the syntax of the language:

$$e ::= \cdots \mid \mathsf{error} \mid \mathsf{try}\ e_1\ \mathsf{with}\ e_2$$

We do not add try expressions to our evaluation contexts—doing so would allow exceptions to "jump over" handlers. Instead, we add a special propagation rule for try:

$$\frac{e_1 \longrightarrow e_1'}{\mathsf{try}\ e_1\ \mathsf{with}\ e_2 \longrightarrow \mathsf{try}\ e_1'\ \mathsf{with}\ e_2}$$

and rules for propagating and catching exceptions. (The side condition $E \neq [\cdot]$ ensures that error by itself does not step.)

$$\frac{E \neq [\cdot]}{E[\mathsf{error}] \longrightarrow \mathsf{error}} \qquad \frac{}{\mathsf{try}\ \mathsf{error}\ \mathsf{with}\ e \longrightarrow e} \qquad \frac{}{\mathsf{try}\ v\ \mathsf{with}\ e \longrightarrow v}$$

The typing rule for exceptions allows them to take *any* type, while the typing rule for try-with expressions requires both sub-expressions to have the same type:

$$\frac{}{\Gamma \vdash \mathsf{error} : \tau} \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \mathsf{try}\ e_1\ \mathsf{with}\ e_2 : \tau}$$

The first typing rule is extremely flexible, allowing errors to be thrown anywhere in a program. Note that it breaks the uniqueness of types: for example, both $\vdash \mathsf{error} : \mathbf{int}$ and $\vdash \mathsf{error} : \mathbf{bool}$ hold. However, it is not hard to see that it causes the progress lemma to become false: the expression error is not a value but is stuck. Fortunately, we can prove the following weaker version, which is still strong enough to prove a useful form of type soundness.

Lemma (Progress)**.** *If $\vdash e : \tau$ then e is a value or e is* error *or there exists e' such that $e \longrightarrow e'$.*

The preservation lemma remains unchanged.
The actual soundness theorem is as follows:

Theorem (Soundness)**.** *If $\vdash e : \tau$ and $e \longrightarrow^* e'$ and $e' \not\longrightarrow$ then either e' is a value or e' is* error*.*